LAB SESSION 10

Doubly Linked List – Insertion and Traversal

THEORY

A Doubly Linked List (DLL) is a linked data structure where each node stores two pointers — one pointing to the previous node and the other pointing to the next node. This allows movement in both forward and backward directions, offering more flexibility than singly linked lists.
In a DLL, nodes are dynamically allocated and can be inserted or removed without shifting elements, as in arrays. The cost of this flexibility is slightly higher memory usage due to the extra pointer.

Node Structure:

1. prev – Address of the previous node (NULL for the first node).
2. data – The stored information.
3. next – Address of the next node (NULL for the last node).

Insertion Methods:

- **At Beginning** – Link the new node's next to the current head and set head's prev to the new node, then update head.
- **At End** – Link the new node's prev to the last node and last node's next to the new node.
- **At Position** – Adjust prev and next pointers of surrounding nodes to fit the new node in between.

Advantages:

- Traversal possible in both directions.
- Faster insertion/deletion at both ends.

Disadvantages:

- Requires more memory per node.
- Slightly more complex pointer management.

Applications:

- Undo/redo in applications.
- Browser navigation history.
- Deque implementation.

PROCEDURE

1. Define a Node structure with data, prev, and next pointers.
2. Initialize head as NULL.
3. Implement:
4. insertAtBeginning()
5. insertAtEnd()
6. insertAtPosition()
7. Implement a displayForward() function for forward traversal.
8. Implement a displayBackward() function for reverse traversal.
9. Test all insertion operations with sample data.

CODE

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* prev;
    Node* next;
};

Node* head = NULL;

void insertAtBeginning(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;
    if (head != NULL)
        head->prev = newNode;
    head = newNode;
}

void insertAtEnd(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}

void insertAtPosition(int value, int pos) {
    if (pos == 0) {
        insertAtBeginning(value);
        return;
    }
    Node* temp = head;
    for (int i = 0; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        cout << "Position out of range.\n";
        return;
    }
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = temp->next;
```

```
    newNode->prev = temp;
    if (temp->next != NULL)
        temp->next->prev = newNode;
    temp->next = newNode;
}

void displayForward() {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

void displayBackward() {
    Node* temp = head;
    if (temp == NULL) return;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->prev;
    }
    cout << endl;
}

int main() {
    insertAtBeginning(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtPosition(15, 1);
    cout << "Forward: ";
    displayForward();
    cout << "Backward: ";
    displayBackward();
    return 0;
}
```

EXPECTED OUTPUT

```
Forward: 10 15 20 30
Backward: 30 20 15 10
```

EXERCISE

1. Modify insertAtPosition() to handle inserting at the last position without using insertAtEnd().
2. Write a function to count the total number of nodes in the DLL.
3. Write a function to reverse the DLL in place.